NATIONAL AI HACKATHON 2026 · SCHUBKI · MAKING SENSE OF LOGS

One corpus. Three roles. Every answer has a receipt.

Just under 5 million real log lines from a supercomputer and a cloud platform. An engineer, a team lead, and a manager each need a different answer from the same logs. Most AI tools over logs either dump raw text back at you or confidently invent the parts they don't know.

ROLE A

ENGINEER

What failed, in which component, and what happened just before and after?

ROLE B

TEAM LEAD

Which problems recur, how often, and which team owns them?

ROLE C

MANAGER

Which observed problems deserve attention first, with no invented impact?

01 THE IDEA

The database searches and calculates. Claude explains. Every number traces to a query that actually ran.

Incident Lens is an MCP server: 13 parameterized tools over BGL and OpenStack logs in SereneDB. Claude never writes SQL; the server does all searching and counting, and writes every call to a hash-chained evidence ledger before returning. Any Claude, yours, a judge's, anyone's, can connect and use it with their own account, from a terminal, Claude Desktop, or a web console built for this.

LEDGER ENTRY

E-0097 tool: search_logs

```
scope: bgl · 2005-06-01 → 2006-02-01
filters: query="invalid or missing
  program image", mode=phrase
total_matches: 112584  truncated: true
ledger_hash: 97633d5663bf9ef0…
note: "quote frequency only from
  total_matches, never len(rows)"
```

02 ARCHITECTURE

Claude sees only what the server returns



The read-only role `il_reader` can never write; every tool call is logged before it is returned, so any number a judge sees can be replayed with `get_evidence` and independently re-verified with `verify_ledger_integrity`.

03 ONE REAL ISSUE · THREE LENSES · BGL

ciod job-launch failures: not a textbook error, a real discovery

A parallel job's executable path is missing or unreadable. Found, traced, and quantified by the same three tool paths each role would actually use.

A

ENGINEER

First hit: job 65641 ("raptor") fails on 1,024 distinct nodes within milliseconds of each other. Not one bad node.

`search_logs → nearby_events → count_by`

B

TEAM LEAD

112,584 entries, 52,763 episodes. One week (2005-08-08) alone jumps +9,300%. Owner: Application Support.

`trend → episodes → count_by(team)`

C

MANAGER

Still accelerating (+224% in the last 4 weeks) while two other top-ranked problems dropped to zero. Priority score: 19,502 vs 0 vs 0.

`corpus_overview → recurring_patterns`

CONTROL

BLIND-LLM CHECK

Asked the identical question, a fresh Claude with no tools and no data access confidently named memory/ECC errors on KERNEL as the top cause, with invented frequency numbers, then admitted, unprompted, that every specific claim was "a pure guess." Wrong component, wrong category, wrong everything except the fluency.

04 A SECOND, DELIBERATELY DIFFERENT SHAPE · OPENSTACK

The methodology isn't a one-dataset trick

A nova-compute periodic task (image-cache verification) throws and aborts entirely whenever it hits one already-deleted instance's disk. A real code-level gap, not an environmental blip.

E1

8

occurrences, over ~26 hours

E2

8

separate episodes (entries = episodes, the opposite shape from the BGL burst above)

E3

8

different VM instances, never the same one twice

E4

#1

most frequent ERROR-level template in the whole OpenStack corpus

Same tool-calling loop, same four-block answer contract, same evidence ledger, run against a completely different system with a completely different failure shape.

`investigation/openstack_investigation.md`, checked clean against the citation checker.

05 NOVELTY · A TOOL THE HACKATHON DIDN'T ASK FOR

What a frequency ranking hides, on purpose

`concentration_anomalies` ranks failures by how confined they are to one node or service, not how often they fire. The two rankings disagree, and the disagreement is the finding.

FINDING 1

BURIED BY FREQUENCY

`MACHINE CHECK DCR read timeout`: **14,481 entries, confined to exactly 1 node out of the whole system. Ranked #33 by raw count: invisible past the default view.**

FINDING 2

HIDDEN IN PLAIN SIGHT

`idoproxydb hit ASSERT condition`: **84,168 entries, 1 node. The single most concentrated failure in the whole corpus; a human scrolling a frequency list would never isolate it this cleanly.**

A high ratio says "go check that one node's hardware first." A low ratio says "this is probably systemic." Both are exact counts. No ML, no guessing.

06 SERENEDB AT SCALE

4,955,757 rows. Every number below actually scanned them.



LOAD TIME

Load + index the full corpus: 3.4–9.2s tables, 5.9–9.6s inverted indexes, across repeated runs on a laptop. No pre-aggregation, no caching tricks.

ROOT CAUSE

The one outlier we found (`search_logs`, relevance mode) was root-caused and fixed, not hidden: a redundant time filter was silently disabling the index's fast top-k scoring path. Dropping the provably-redundant filter turned a ~1.5s query into ~150-215ms - same result, 9x faster.

07 THE EVIDENCE LEDGER & ANSWER CONTRACT

Four blocks. Every number carries a receipt, and the receipt book is tamper-evident.

BLOCK 1

1 · OBSERVED

Measured facts only: scope, period, filters, `[E-id]`. Never `len(rows)` as a frequency.

BLOCK 2

2 · HYPOTHESES

Possible explanations, explicitly labelled. Never stated as fact.

BLOCK 3

3 · RECOMMENDED CHECKS

Specific, ordered, each tied to an evidence id.

BLOCK 4

4 · UNKNOWN

What the logs don't establish: cause, downtime, cost, customer impact. Never invented.

INTEGRITY CHECK

TAMPER-EVIDENT, NOT JUST APPEND-ONLY

Every ledger entry's hash is `sha256(previous entry's hash + this entry's own content)`. Hand-edit one stored number with a raw SQL `UPDATE`, bypassing every tool, and `verify_ledger_integrity` catches it and names the exact entry. Proven live in the web console's "Simulate Tamper" button, against a scratch copy that never touches the real ledger.

08 A BROWSER YOU CAN WATCH IT THINK IN

The same 13 tools, with the evidence ledger made visible

UI

Chat with Claude through the browser: same role prompts, same tools, called through the real Anthropic API. Every tool call lands on the right as it happens: the exact SQL, the exact count, the ledger hash, chained visibly to the one before it. Click any evidence id to replay it.

TEST COVERAGE

Not a second implementation. It's a thin UI over the exact same server. Tested over the real HTTP surface (overview, ledger status, tamper demo, evidence replay, path-traversal blocked), and the chat loop itself is driven end-to-end by a fake-client test harness, so the only genuinely untested piece is the literal network call to Anthropic, which needs a real key to exercise once, same as any MCP client.

09 BONUS · DATABASE ROW-LEVEL SECURITY

Per-team access, enforced by the database, not application code

SereneDB has no native `ROW LEVEL SECURITY` / `CREATE POLICY`. checked directly, not assumed. Built the same real guarantee from primitives it does support: one filtered view per team, straight from the ownership table, with that team's base-table grants revoked.

F1

9

real teams provisioned, straight from the ownership table

F2

0

rows any team's role can see outside its own component/service

F3

✓

verified by connecting as each restricted role, never as admin

10 SCOREBOARD

What's actually been run, not just written

01

13

MCP tools, all wired and tested

02

39

pytest tests against plain-SQL ground truth

03

4.9M

real log rows loaded, indexed, and queried

04

15+

real bugs found and fixed by testing at full scale, not assumed away

05

2

full worked investigations, both datasets, both clean against the citation checker

06

9

per-team database roles, each verified in isolation

07

1

clean-checkout run, fresh venv to passing test suite, zero manual fixes

08

0

invented financial, downtime, or customer-impact claims

11 LIMITS & WHAT'S NEXT

Honest about what the logs can't tell you

EXCLUDED

DOES NOT ESTABLISH

Root cause of any single job's bad path · who submitted the spike · downtime caused · financial or customer impact.

PENDING

NEXT UP

A live golden-set run against the real Claude API, same as any MCP client needs its own key for.

github.com/ApparentlyTejas/incident-lens

One setup command. Clean checkout to first answered question in minutes, verified, not assumed.